Herramientas de GitHup

Presentado por; Anny Sofia Beltrán García

Presentado a; Carlos Andrés Gómez Vasco

Medicina veterinaria

Universidad agraria de Colombia

UNIAGRARIA

Ciencia de datos

2026

CAPÍTULO 7. HERRAMIENTAS DE GIT

Introducción

Git cuenta con diferentes herramientas que permiten realizar tareas más avanzadas durante el desarrollo de un proyecto. Estas herramientas ayudan a revisar versiones anteriores, organizar modificaciones, guardar temporalmente el trabajo, corregir errores y administrar el historial de cambios. El conocimiento de estas funciones permite aprovechar mejor Git, ya que no solamente se utiliza para guardar archivos, sino también para controlar de manera detallada el desarrollo de un proyecto y solucionar situaciones que pueden presentarse durante el trabajo individual o colaborativo

7.1 Revisión por selección

La revisión por selección permite identificar y consultar versiones específicas de un proyecto. Git utiliza identificadores de confirmaciones o commits para reconocer cada estado guardado del repositorio. Para consultar el historial de cambios se utiliza el comando **git log**, que muestra las confirmaciones realizadas, sus autores, fechas y mensajes. También se puede utilizar **git show** para observar los detalles de una confirmación específica

- **Entre los comandos más utilizados se encuentran:**
- **git log:** muestra el historial de confirmaciones
- **git show**: presenta los detalles de una confirmación
- **git log --oneline**: muestra el historial de forma resumida
- **git diff:** permite comparar modificaciones entre archivos o versiones

7.2 Organización interactiva

La organización interactiva permite seleccionar de manera detallada qué modificaciones se desean incluir en una confirmación. Esto resulta útil cuando se han

realizado varios cambios en un mismo archivo, pero solamente algunos deben guardarse en un commit. Para utilizar esta herramienta se emplea el comando **git add -i,** que abre un menú interactivo con diferentes opciones para organizar los cambios. También se puede utilizar **git add -p** para seleccionar partes específicas de las modificaciones

- **Esta función permite:**
- Seleccionar los cambios que se desean guardar
- Separar modificaciones de un mismo archivo
- Evitar incluir cambios que todavía no están terminados
- Organizar mejor las confirmaciones del proyecto

7.3 Guardado rápido y limpieza

Git permite guardar temporalmente los cambios realizados en un proyecto sin necesidad de crear una confirmación. Esta función se conoce como stash y es útil cuando se necesita cambiar de rama o realizar otra actividad sin perder el trabajo que todavía no está terminado. El comando principal es **git stash**, que guarda temporalmente las modificaciones.

- **Posteriormente, se pueden recuperar mediante diferentes comandos:**
- **git stash**: guarda temporalmente los cambios
- **git stash list**: muestra los cambios guardados
- **git stash apply**: recupera los cambios sin eliminarlos de la lista
- **git stash pop**: recupera los cambios y los elimina de la lista si se aplican correctamente
- **git stash drop:** elimina un guardado temporal específico

Por otra parte, Git cuenta con el comando **git clean**, que permite eliminar archivos que no están siendo controlados por el repositorio. Es importante utilizarlo con precaución, ya que puede borrar archivos que no se encuentran guardados en Git. Para revisar qué archivos se eliminarían antes de ejecutar la limpieza se puede utilizar **git clean -n**

7.4 Firmando tu trabajo

Git permite firmar digitalmente las confirmaciones y etiquetas para comprobar su autenticidad. Esta herramienta ayuda a verificar que una modificación fue realizada por una persona que cuenta con una clave de firma reconocida. La firma digital puede utilizarse mediante tecnologías como GPG o SSH, dependiendo de la configuración del usuario

Algunos comandos relacionados son:

- **git commit -S -m "Mensaje":** crea una confirmación firmada
- **git tag -s nombre_etiqueta -m "Descripción":** crea una etiqueta firmada
- **git verify-commit**: verifica la firma de una confirmación
- **git verify-tag**: comprueba la firma de una etiqueta

Para utilizar estas funciones es necesario configurar previamente el sistema de firma correspondiente. La firma digital aporta mayor confianza y seguridad en proyectos donde es importante verificar el origen de los cambios

7.5 Buscando

Git cuenta con herramientas que permiten buscar información dentro de los archivos y del historial de un proyecto. Estas funciones son útiles para localizar palabras, fragmentos de código, modificaciones y errores

- **Entre los comandos principales se encuentran:**

- **git grep "palabra"**: busca una palabra o expresión dentro de los archivos del proyecto
- **git log --grep="texto":** busca confirmaciones cuyos mensajes contienen un texto determinado
- **git log -S"texto":** localiza confirmaciones donde cambia la cantidad de apariciones de un texto específico
- **git blame nombre_archivo**: muestra quién modificó cada línea de un archivo y en qué confirmación se realizó el cambio

Permiten encontrar información de manera rápida y conocer el origen de las modificaciones realizadas en un proyecto

7.6 Reescribiendo la historia

Git permite modificar el historial de confirmaciones para corregir errores o reorganizar el trabajo. Esta función se conoce como reescritura de la historia y debe utilizarse con cuidado, especialmente cuando los cambios ya fueron compartidos con otros colaboradores. Una de las herramientas utilizadas es **git commit --amend**, que permite modificar la última confirmación. Por ejemplo, se puede corregir un mensaje o incluir archivos que fueron olvidados. También existe el comando **git rebase**, que permite reorganizar las confirmaciones de una rama sobre otra base. Esto ayuda a mantener un historial más ordenado, aunque puede cambiar los identificadores de las confirmaciones. Es importante recordar que modificar confirmaciones que ya fueron compartidas puede generar dificultades para otros colaboradores. Por esta razón, se recomienda evitar reescribir el historial público sin coordinación con el equipo

7.7 Reiniciar desmitificado

El comando git reset permite modificar el estado de la rama actual y controlar qué cambios permanecen preparados para ser confirmados. Es una herramienta importante para corregir errores durante el trabajo con Git

- **Existen tres modalidades principales:**
- **git reset --soft**: mueve la referencia de la rama, pero conserva los cambios preparados para confirmación
- **git reset --mixed**: mueve la referencia y retira los cambios del área de preparación, conservándolos en los archivos de trabajo
- **git reset --hard**: mueve la referencia y descarta los cambios preparados y los cambios locales afectados

El uso de **git reset --hard** requiere especial precaución porque puede eliminar modificaciones que todavía no se han guardado en una confirmación

7.8 Fusión avanzada

La fusión avanzada permite integrar cambios de diferentes ramas cuando el proyecto presenta situaciones más complejas. Git compara las modificaciones y busca combinarlas para conservar el trabajo realizado por los colaboradores. Cuando dos ramas modifican las mismas partes de un archivo, pueden aparecer conflictos que deben resolverse manualmente. El usuario debe revisar las diferencias, decidir qué contenido conservar y guardar el resultado de la fusión

- **Algunos comandos relacionados son:**
- **git merge nombre_rama**: integra los cambios de una rama en la rama actual
- **git merge --abort**: cancela una fusión que presenta conflictos antes de finalizarla

- **git diff**: permite revisar las diferencias entre archivos y versiones
- **git status**: muestra el estado del repositorio y los archivos que presentan conflictos

7.9 Rerere

Rerere es una herramienta de Git cuyo nombre proviene de la expresión inglesa reuse recorded resolution, que significa reutilizar resoluciones registradas. Su función consiste en recordar cómo se resolvió anteriormente un conflicto de fusión para poder aplicar esa misma solución cuando el conflicto vuelva a presentarse.

- Para activar esta herramienta se utiliza el comando**: git config --global rerere.enabled true.**

7.10 Haciendo debug con Git

Git dispone de herramientas que permiten identificar errores introducidos durante el desarrollo de un proyecto. Una de ellas es el comando git bisect, que utiliza un proceso de búsqueda binaria para encontrar la confirmación que introdujo un problema.

El procedimiento consiste en indicar una versión donde el proyecto funciona correctamente y otra donde aparece el error. Git selecciona versiones intermedias para que el usuario compruebe si el problema está presente

- **Entre los comandos utilizados se encuentran:**
- **git bisect start**: inicia el proceso de búsqueda
- **git bisect bad**: identifica una versión donde existe el error
- **git bisect good**: identifica una versión donde el proyecto funciona correctamente
- **git bisect reset**: finaliza el proceso y regresa al estado anterior

Esta herramienta permite localizar errores de manera más eficiente cuando se trabaja con proyectos que tienen un historial extenso de modificaciones

7.11 Submódulos

Los submódulos permiten incorporar un repositorio Git dentro de otro repositorio. Esta función es útil cuando un proyecto depende de otro proyecto externo o necesita utilizar componentes que se administran de manera independiente

- Para agregar un submódulo se utiliza el comando: **git submodule add URL_DEL_REPOSITORIO**. De esta manera, el proyecto principal registra la ubicación y la referencia del repositorio incorporado

  Otros comandos importantes son:

- **git submodule init**: inicializa la configuración de los submódulos
- **git submodule update**: descarga o actualiza el contenido correspondiente
- **git submodule update --init --recursive**: inicializa y actualiza los submódulos, incluyendo los que contienen otros submódulos

Los submódulos permiten reutilizar componentes y mantener separados los proyectos relacionados

7.12 Agrupaciones

Git permite crear archivos de agrupación o bundles, que contienen información del repositorio y pueden utilizarse para transferir cambios sin depender de una conexión directa entre repositorios. Esta herramienta resulta útil cuando se necesita compartir un proyecto mediante un archivo, por ejemplo, cuando no existe acceso a internet o cuando se desea transportar una copia del historial

- Para crear un archivo de agrupación se utiliza el comando: **git bundle create nombre_archivo.bundle --all.**

- Para comprobar su contenido se puede utilizar: **git bundle list-heads nombre_archivo.bundle**.

Los bundles facilitan el intercambio de información entre repositorios y permiten conservar el historial de trabajo dentro de un archivo

7.13 Replace

La función replace permite sustituir temporalmente la referencia de un objeto de Git por otro objeto. Esto puede utilizarse para modificar la manera en que Git presenta determinados elementos del historial sin cambiar directamente los objetos originales. Se utiliza mediante el comando **git replace**, que permite administrar estas referencias de reemplazo. Esta herramienta está orientada a situaciones avanzadas, como la manipulación y consulta de historiales complejos. Su utilización requiere conocer cómo Git almacena los objetos y las referencias dentro de un repositorio

7.14 Almacenamiento de credenciales

Git necesita utilizar credenciales para autenticar al usuario cuando se conecta con determinados repositorios remotos. Para facilitar este proceso existen herramientas que permiten almacenar o administrar las credenciales de acceso. El comando git config --global credential.helper permite configurar el mecanismo que Git utilizará para gestionar las credenciales. En algunos sistemas se pueden utilizar administradores de credenciales que almacenan la información de manera protegida y facilitan las conexiones posteriores. Es importante utilizar mecanismos seguros y evitar guardar contraseñas directamente en archivos de texto sin protección. El almacenamiento adecuado de credenciales ayuda a proteger las cuentas y los repositorios

7.15 Ejercicio práctico: utilizar herramientas de Git para revisar y recuperar cambios

- **Procedimiento:**

1. Abrir Git Bash e ingresar a la carpeta de un repositorio de prueba mediante el comando: **cd nombre_del_repositorio**
2. Consultar el historial de confirmaciones utilizando: **git log --oneline**
3. Crear un archivo llamado prueba.txt y escribir un contenido sencillo
4. Agregar el archivo al área de preparación mediante; **git add prueba.txt**
5. Crear una confirmación utilizando: **git commit -m "Creación del archivo de prueba"**
6. Modificar el contenido del archivo prueba.txt y guardar los cambios
7. Utilizar: **git diff** para observar las modificaciones realizadas
8. Guardar temporalmente los cambios con **git stash**
9. Consultar los guardados temporales mediante: **git stash list**
10. Recuperar los cambios utilizando: **git stash pop**
11. Buscar una palabra dentro del repositorio: con **git grep "palabra"**
12. Consultar nuevamente el historial mediante: **git log --oneline** para verificar las confirmaciones registradas

Se podrá observar cómo Git permite revisar modificaciones, guardar temporalmente el trabajo y buscar información dentro del proyecto

Conclusiones

Las herramientas avanzadas de Git permiten administrar los proyectos de una manera más completa y organizada. Mediante funciones como stash, reset, rebase, bisect y grep es posible recuperar cambios, revisar el historial, encontrar errores y controlar las

modificaciones realizadas. Además, herramientas como los submódulos, las firmas digitales y el almacenamiento de credenciales amplían las posibilidades de Git para trabajar con proyectos más complejos y colaborativos. El conocimiento de estas funciones permite solucionar problemas frecuentes durante el desarrollo y mejorar el manejo del control de versiones en proyectos académicos y profesionales

Referencias bibliográficas

1. Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2
2. Git. (s. f.). Documentación de Git. https://git-scm.com/docs
3. GitHub. (s. f.). GitHub Docs. https://docs.github.com/es